Module 10: Serving Systems

Scheduling Tokens under Latency and Memory Constraints

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. From model forward to inference engine
2. Token and memory scheduling
3. Measure the service, not a toy loop

Guiding question

How does an inference engine turn irregular requests into efficient GPU work while protecting latency objectives?

What should be clear by the end?

1. Trace a request through scheduler, cache manager, model runner, and output path.
2. Explain continuous batching and chunked prefill at iteration granularity.
3. Connect paged KV memory, admission control, and preemption.
4. Design a serving benchmark with explicit workload and latency metrics.

PART 1

From model forward to inference engine

1. From model forward to inference engine
2. Token and memory scheduling
3. Measure the service, not a toy loop

A serving engine coordinates four state machines

1. request queue and admission policy
2. iteration-level token scheduler
3. KV-block allocator and prefix index
4. model runner and sampled output stream

One engine step

select tokens
assign cache blocks
execute model
sample tokens
update request state

Source: vLLM Anatomy and Hugging Face Continuous Batching

Prefill and decode compete for the same device

Prefill request

many prompt tokens
high parallelism
large temporary attention work
drives time to first token

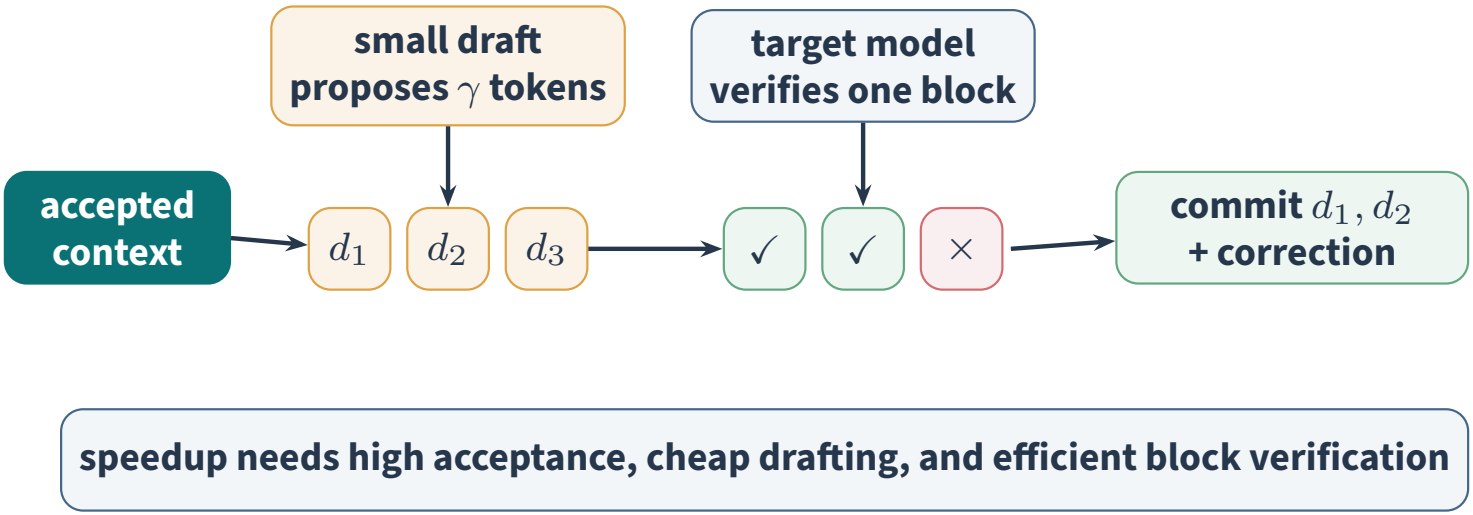
Decode request

one token per active sequence
growing KV reads
latency sensitive
drives inter-token latency

Scheduling tension

A long prefill can increase throughput but block many decode tokens, causing visible latency spikes.

Speculative decoding



Correct rejection sampling preserves the target distribution; measure accepted tokens per target step and TPOT.

Source: MVA Training and Deploying Large-Scale Models, Lecture 6 (2026)

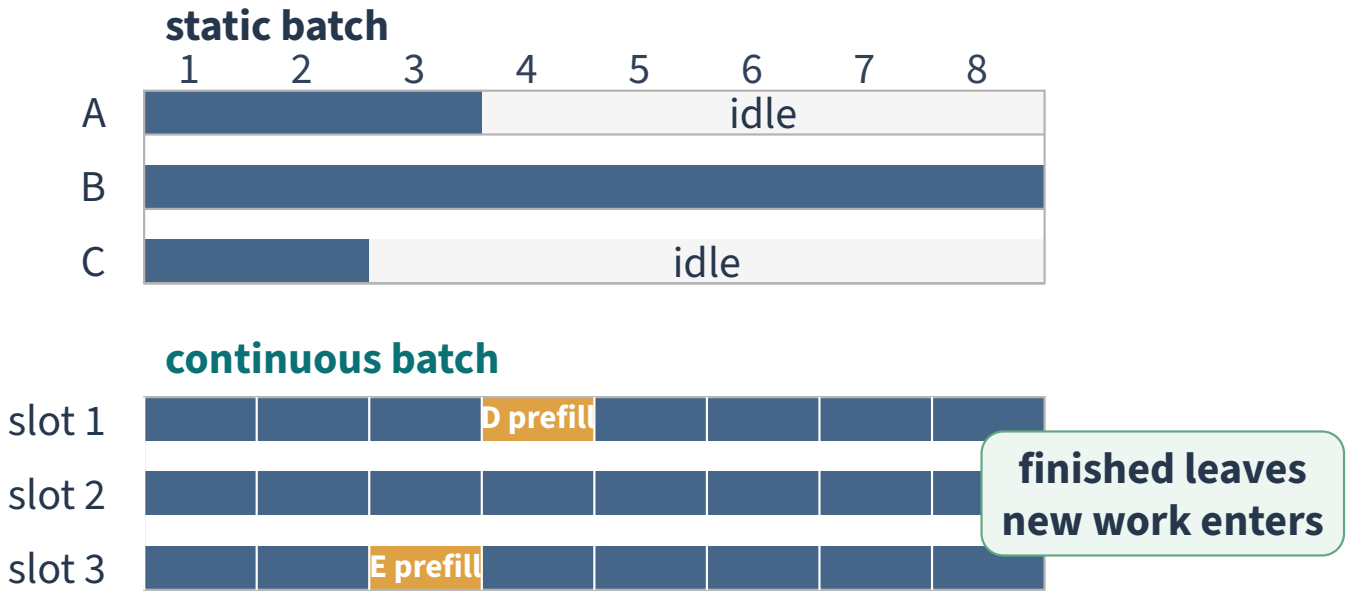
Static batching waits for the slowest request



Head-of-line effect

Completed sequences occupy batch slots until the longest member finishes. New work cannot enter even when capacity is available.

Continuous batching repacks every iteration



Scheduling unit

Repack useful decode tokens and prefill chunks every step, subject to token and KV-block budgets.

PART 2

Token and memory scheduling

1. From model forward to inference engine
2. Token and memory scheduling
3. Measure the service, not a toy loop

A token budget bounds work per engine step

Suppose the iteration budget is $N_{\max}$ scheduled tokens.

$$N_{\text{decode}} + N_{\text{prefill}} \leq N_{\max}$$

- Every active decode sequence usually asks for one token.
- Remaining capacity can admit prompt tokens.
- Cache-block availability may be the tighter constraint.

Policy matters

Prioritizing decode protects inter-token latency. Prioritizing large prefills may improve throughput but delay active users.

Chunked prefill turns long prompts into schedulable pieces

Instead of scheduling a 16k-token prompt atomically, split it into chunks that share iterations with decode work.

Benefits

lower head-of-line blocking
bounded iteration size
better prefill-decode mixing

Costs

more scheduling boundaries
possible loss of prefill efficiency
partial-request state

Chunk size is a latency-throughput knob, not a universal constant.

Paged KV allocation matches irregular lifetimes

- Divide KV memory into fixed-size physical blocks.
- Map each request's logical token blocks to any free physical blocks.
- Allocate as the sequence grows and return blocks on completion.
- Share immutable prefix blocks when requests have identical prefixes.

What paging solves

External fragmentation and maximum-length over-reservation are reduced because requests need not occupy one contiguous region.

What remains

Internal slack in the last block, allocator metadata, capacity limits, and bandwidth cost of reading the cache.

Prefix caching trades index work for avoided prefill

A cache hit can reuse KV blocks for an identical token prefix.

Works well for

repeated system prompts
shared document prefixes
branching completions

Correctness conditions

identical model and adapter
identical token IDs and positions
compatible cache precision and layout

Measure hit rate, tokens avoided, retained-block cost, and eviction behavior together.

Admission control prevents memory collapse

Before admitting work, estimate required blocks from prompt length, current cache, and maximum generation policy.

- queue when safe cache capacity is unavailable
- reject or cap requests outside the service contract
- preempt lower-priority work only under a declared policy
- distinguish recomputation preemption from host-memory swapping

Overcommit is not free

Preemption can recover capacity but adds wasted compute, extra latency, and tail-risk amplification.

CUDA graphs reward stable execution shapes

Capturing a repeated GPU execution removes CPU launch overhead, but dynamic batches change shapes and memory addresses.

Engine strategy

capture a set of padded batch shapes
reuse stable buffers
fall back for unsupported shapes

Trade-off

More capture sizes reduce padding but consume memory and operational complexity.

Checkpoint: schedule by hand

Think, then compare

An iteration budget is 16 tokens. Ten sequences need decode, one waiting prompt has length 20, and cache capacity is available. Propose the next two iterations under a decode-first chunked-prefill policy.

PART 3

Measure the service, not a toy loop

1. From model forward to inference engine
2. Token and memory scheduling
3. Measure the service, not a toy loop

Latency is a distribution with several clocks

Metric	Definition	Dominant factors
queue time	arrival to admission	load and policy
TTFT	arrival to first token	queue plus prefill
inter-token latency	time between output tokens	decode schedule and batch
end-to-end latency	arrival to completion	all phases and output length
throughput	completed or output tokens per second	batching and saturation

Report median and tail percentiles. A good average can hide an unusable p99.

Open-loop and closed-loop loads answer different questions

Closed loop

Each client waits for completion before sending again. Offered load falls automatically when the server slows.

Open loop

Arrivals follow an external process. Queues grow when demand exceeds capacity, revealing overload behavior.

Benchmark declaration

Report arrival process, prompt and output distributions, sampling settings, concurrency, warmup, and duration.

A useful benchmark sweeps the operating curve

1. choose a representative request-length distribution
2. increase arrival rate or concurrency gradually
3. record TTFT, inter-token latency, throughput, cache occupancy, and preemptions
4. identify the saturation knee and tail-latency cliff
5. compare policies at matched service-level objectives

The right conclusion

“Policy A delivers more tokens/s while keeping p99 TTFT below the declared threshold for workload W.”

Read metrics as one coupled system

- rising queue time with stable GPU time means admission is saturated
- TTFT spikes with stable decode latency suggest long prefills or queueing
- growing preemptions suggest cache overcommit
- high cache occupancy with low throughput suggests bandwidth or scheduler inefficiency
- lower mean latency with worse p99 may indicate unfair prioritization

Checkpoint: serving report

Think, then compare

Define a workload, one service-level objective, the policy under test, a baseline, and the five measurements needed to explain both throughput and tail latency.

Key takeaways

1. An inference engine schedules token work and cache state at every iteration.
2. Continuous batching removes static slots, while chunked prefill controls head-of-line blocking.
3. Paged memory, prefix reuse, admission, and preemption jointly determine capacity.
4. Serving claims require representative arrivals plus throughput and tail-latency evidence.